# Software Engineering - Week 1 Lecture

*Introduction to Software Engineering | Duration: 1.5 Hours (90 Minutes)*

**Course:** Software Engineering (SE)

**Lecture Theme:** Understanding software, software engineering, the software crisis, and the role of a software engineer

## Learning Outcomes

By the end of this lecture, students should be able to:

- Define software and software engineering in simple terms.
- Explain why software engineering is needed for large and complex systems.
- Differentiate between programming and software engineering.
- Describe common characteristics of software.
- Explain the software crisis and its major causes.
- Identify the major responsibilities and professional qualities of a software engineer.

## 1.What is Software?

software is not only a mobile application. Software includes programs, data, configuration files, documentation, and related instructions that allow a computer-based system to perform useful tasks.

Software is a collection of computer programs and related data/instructions that tells a computer or device what to do.

## 2. Types of Software

**System Software:** Controls and manages computer hardware. Examples include operating systems, device drivers, and system utilities.

**Application Software:** Helps users perform specific tasks. Examples include word processors, banking applications, e-commerce systems, and student management systems.

**Embedded Software:** Software built into a physical device. Examples include software in cars, washing machines, medical devices, and smart appliances.

**Web and Mobile Software:** Software delivered through browsers or mobile devices, often connected to cloud services and databases.

## 3. Characteristics of Software

- Intangible: Software cannot be physically touched like hardware.
- Developed, not manufactured: Most effort is spent on analysis, design, coding, testing, and maintenance.
- Does not wear out physically: Software usually becomes difficult to maintain because of changes, defects, obsolete technologies, or increasing complexity.
- Highly changeable: Requirements often change after development begins.
- Complex: Large systems may contain thousands or millions of lines of code and many interacting components.
- Maintenance-intensive: A significant part of software work happens after the first release.

## 4. What is Software Engineering?

Software engineering is the disciplined and systematic application of engineering principles, methods, processes, and tools to the development, operation, maintenance, and evolution of software.

In simple words: Software engineering means developing software in an organized, planned, measurable, and quality-focused way rather than simply writing code.

Key idea for students: “Programming is one part of software engineering; software engineering covers the whole software life cycle.”

## 5. Why Do We Need Software Engineering?

- Modern software systems are large and complex.
- Many people may work on the same project.
- Customers may not initially know exactly what they need.
- Requirements change over time.
- Software failures can cause financial loss, safety problems, or reputational damage.
- Projects need predictable cost, schedule, quality, security, and maintainability.
- Organizations need documentation and processes so systems can be maintained by other developers.

## 6. Programming vs. Software Engineering

| **Programming** | **Software Engineering** |
|---|---|
| Main focus is writing code. | Covers the complete software life cycle. |
| Often addresses a specific technical problem. | Addresses requirements, design, development, testing, deployment, and maintenance. |
| Can be performed by one developer. | Often involves teams and stakeholders. |
| May use informal planning for small tasks. | Uses systematic processes, documentation, standards, and project management. |
| Success may mean the program works. | Success also includes quality, security, reliability, maintainability, cost, and user satisfaction. |

## 7. The Software Crisis

The term software crisis describes the difficulties experienced as software systems became larger and more complicated. Organizations faced problems such as projects taking too long, exceeding budgets, containing defects, and becoming difficult to maintain.

Common causes:

- Poor or incomplete requirements.
- Rapidly changing technology and customer needs.
- Underestimation of project size and complexity.
- Weak planning and project management.
- Insufficient testing and quality assurance.
- Poor communication among stakeholders and developers.
- Uncontrolled changes and lack of configuration management.

Teacher example: Imagine a university asks for an online examination system. If developers start coding immediately without discussing roles, question banks, security, grading rules, user authentication, reports, and failure handling, the project may work initially but fail when real users start using it. Software engineering provides methods to reduce these problems.

## 8. Software Engineering Process - Big Picture

Introduce the students to the software life cycle without going into detailed process models yet:

**Requirements:** Understand what the customer and users need.

**Analysis:** Study the problem and organize the requirements.

**Design:** Decide how the system will be structured.

**Implementation:** Write and integrate the software.

**Testing:** Find defects and verify that requirements are satisfied.

**Deployment:** Make the software available to users.

**Maintenance / Evolution:** Fix problems and adapt or improve the system.

## 9. Role of a Software Engineer

- Understand user and business needs.
- Analyze requirements and identify risks.
- Design reliable and maintainable solutions.
- Write and review code.
- Test software and investigate defects.
- Work effectively in teams.
- Document important technical decisions.
- Manage changes and versions.
- Consider security, privacy, safety, usability, and performance.
- Communicate clearly with technical and non-technical stakeholders.

## 10. Professional Ethics

Software engineers often work on systems that affect people's money, education, privacy, health, communication, and safety. Therefore, professional behavior is essential.

**Confidentiality:** Protect information obtained through professional work.

**Competence:** Work within one's knowledge and skills and continue learning.

**Honesty:** Do not misrepresent capabilities, results, costs, or risks.

**Respect for intellectual property:** Follow licenses and copyright requirements.

**Public interest:** Consider the safety and well-being of users and society.

## 11. Class Activity - 10 Minutes

Scenario: A university wants a “Student Attendance Management System.” Divide students into small groups.

- List 5 features the system should provide.
- Identify at least 3 users/actors.
- Identify 2 possible problems if requirements are not collected properly.
- Discuss why simply hiring a programmer may not be enough for a large university system.

## 12. Quick Recap

- Software is more than source code; it includes programs and supporting artifacts.
- Software engineering applies systematic engineering principles to software development and evolution.
- Programming is a component of software engineering, not the entire discipline.
- The software crisis highlights problems caused by increasing software complexity and weak development practices.
- Software engineers must consider requirements, quality, testing, maintenance, teamwork, and ethics.

## 13. Exit Quiz

1. Define software in your own words.

2. What is software engineering?

3. Give two differences between programming and software engineering.

4. What is meant by the software crisis?

5. Name three responsibilities of a software engineer.